Faster Is Never Fast Enough: an HPC look at Bad Software Performance

HAL ELROD

Thesis

- HPC is specialized — but its performance principles are universal
- The HPC mindset can elevate everyday computing

- Hal @ Baylor
  - BA Computer Science and Economics
  - ACM Scholastic Programming Team
  - Econ research, Bear Downs, Honors Program
- After Baylor
  - MSE in Operations Research, U.T. Austin
  - MSCS, Georgia Institute of Technology
  - 35 years in technology leadership, consulting, startups, mostly around competitive energy markets (electricity, natural gas)
  - Instructional Associate for High Performance Computing graduate course (Georgia Tech OMSCS)

Then vs Now: Cray-2 (1985) vs. iPhone 16 Pro

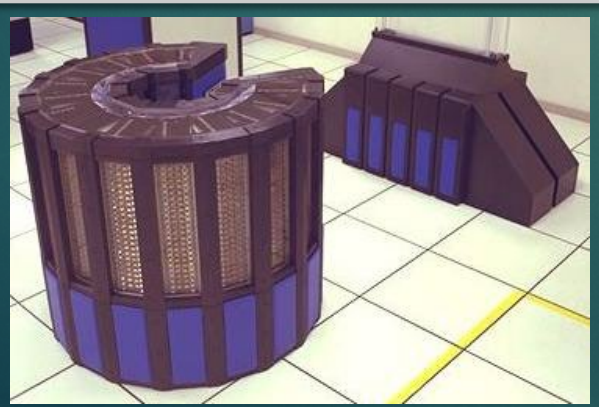
Cray-2 (1985)

Clock speed:	244 MHz
CPU cores:	4 vector processors
RAM:	256 MB
Peak perf.:	1.9 GFLOPS
Storage:	Tape / disk (GBs)
Power draw:	~195 kW
Weight:	~5,000 lbs (2,270 kg)
Price (1985 $):	~$32 million ($100M today)

VS

iPhone 16 Pro (2024)

Clock speed:	~4.0 GHz (P cores)
CPU cores:	6 (2P + 4E)
GPU cores:	6-core Apple GPU
Neural Engine:	16-core NPU
RAM:	8 GB LPDDR5
Peak perf.:	~4.4 TFLOPS (NPU), 2.3 TFLOPS (CPU)
Storage:	Up to 1 TB NVMe
Price (2024 $):	~$999



How can everything now be so fast, but still seem so slow?

High Performance

```
// Not this
foo = foo + 1;
// vs
foo++;

// swap a and b
a ^= b; b ^= a; a ^= b;
```

The Truth

Bit tweaks have limited scope

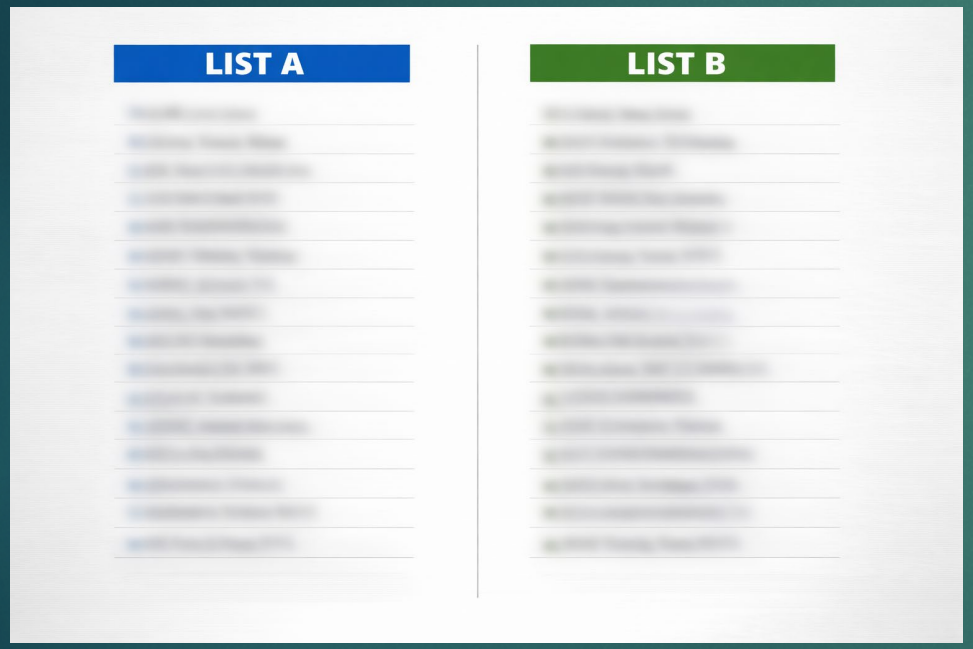
Compilers have gotten really smart

In most cases, just write readable code

"We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil"

Donald Knuth (1974)

Real World Challenge 1



- Compare and match two lists of thousands wholesale energy trades and confirms confirmations across multiple fields
- Consultant's Approach:
  - For each Item from List A
  - Search for corresponding item in List B

What is the computational complexity?

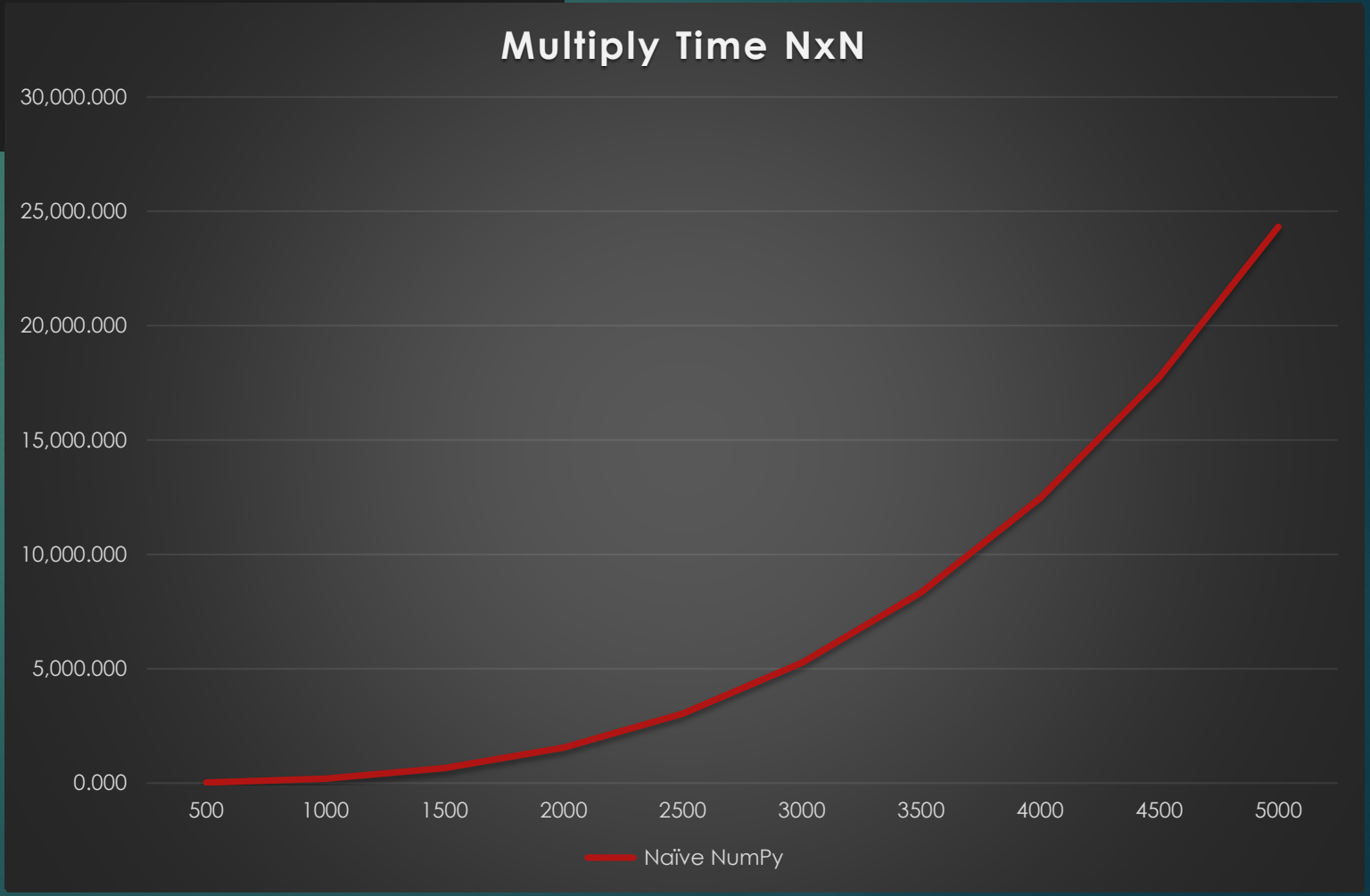
Fun with Matrices

$$\underset{N \times N \text{ Matrix}}{\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ & & \cdots & \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}} \times \underset{N \times N \text{ Matrix}}{\begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & & \cdots & \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}} \longrightarrow \underset{N \times N \text{ Result}}{\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & & \cdots & \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix}}$$

$$c_{i,j} = \sum_{k=1}^{N} a_{ik} \cdot b_{kj}$$

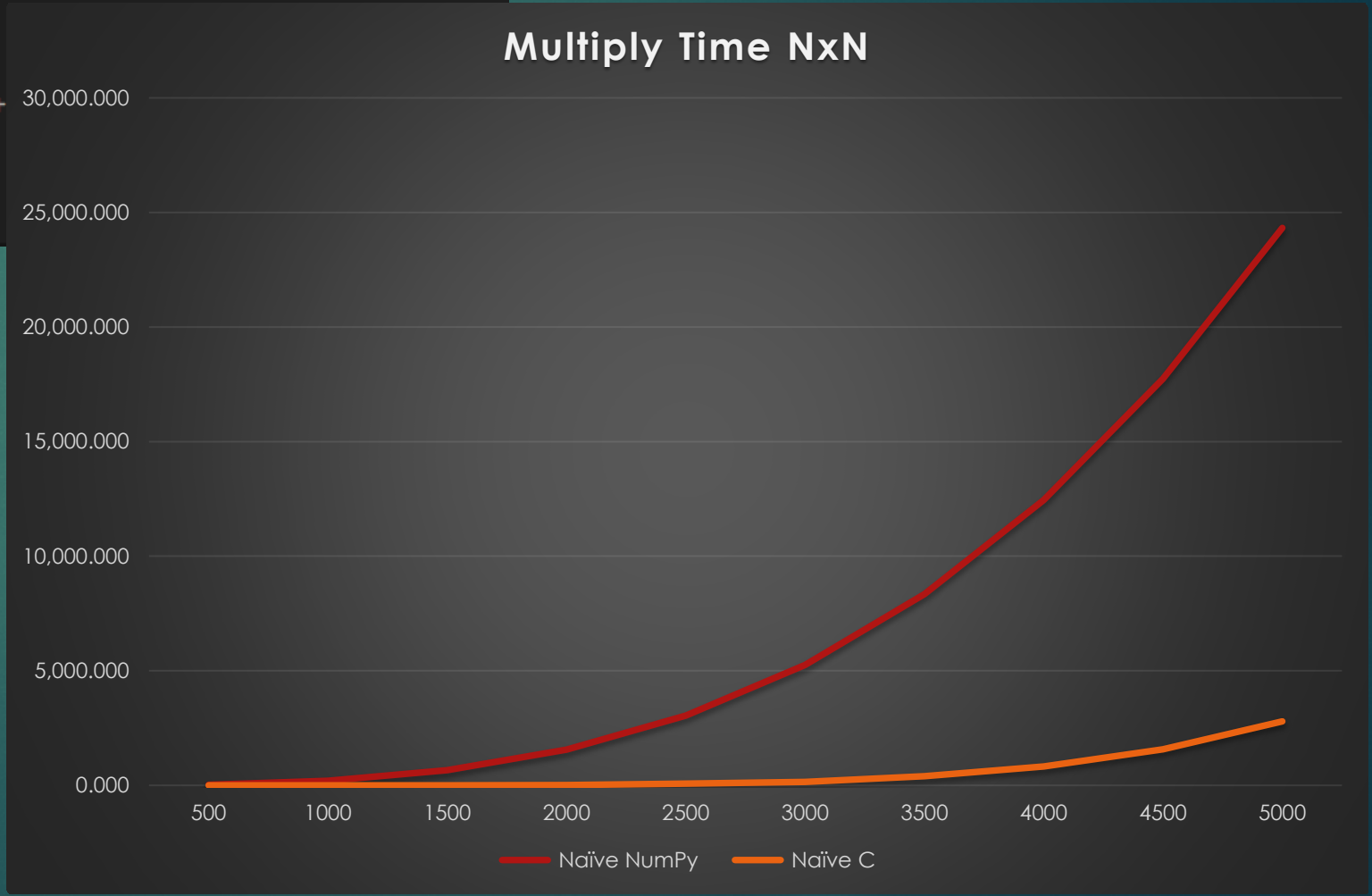
Multiply a Matrix – Naïve Python

```python
def Mmultiply(a: np.ndarray, b: np.ndarray, c: np.ndarray, n: int) -> None:
    for i in range(n):
        for j in range(n):
            s = 0.0
            for k in range(n):
                s += (a[i, k]) * (b[k, j])
            c[i, j] = s
```



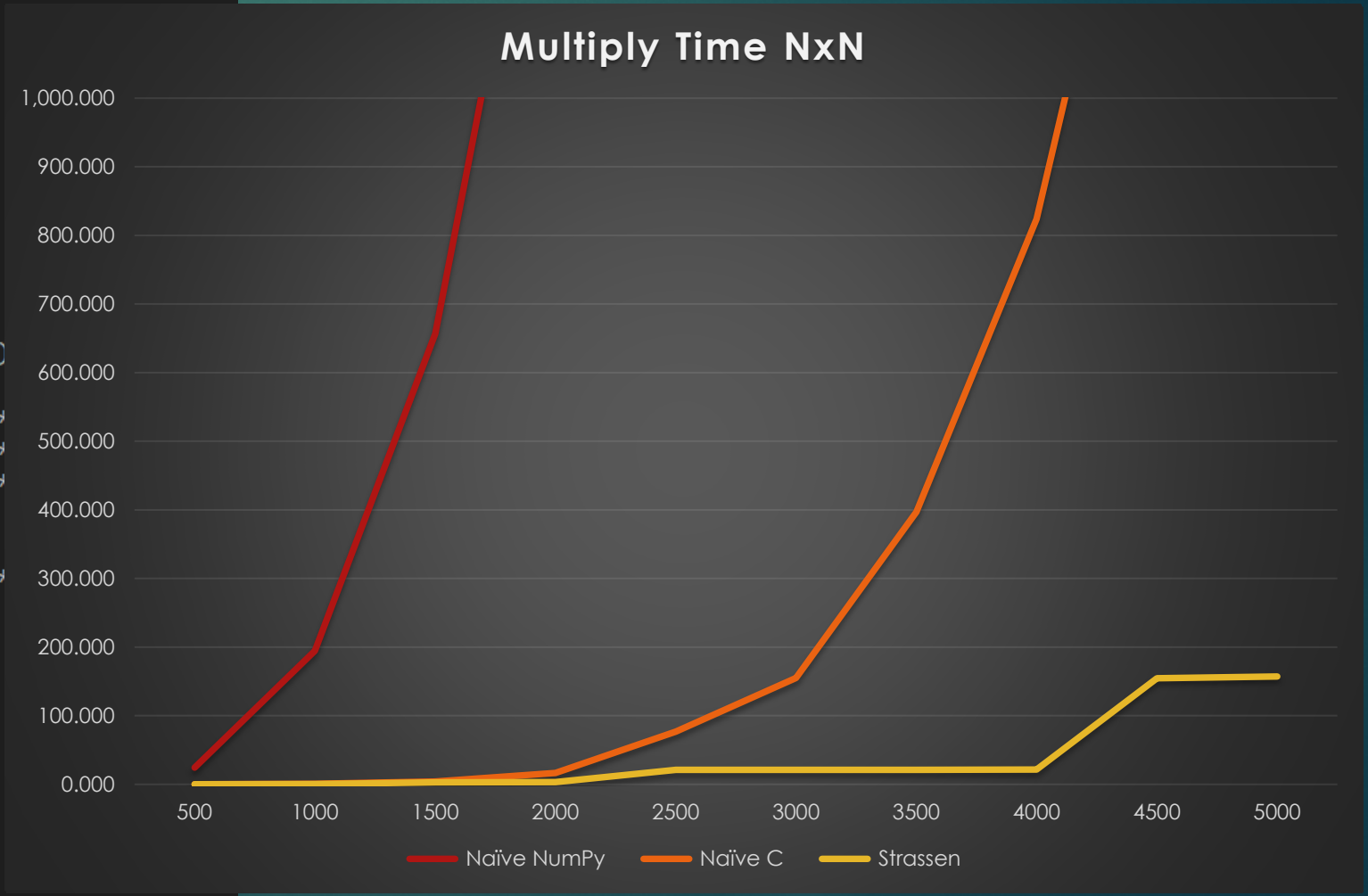
Multiply a Matrix – Naïve C

```
static void multiply(const double *A, const double *B, double *C, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++)
                sum += A[i * n + k] * B[k * n +
            C[i * n + j] = sum;
        }
}
```



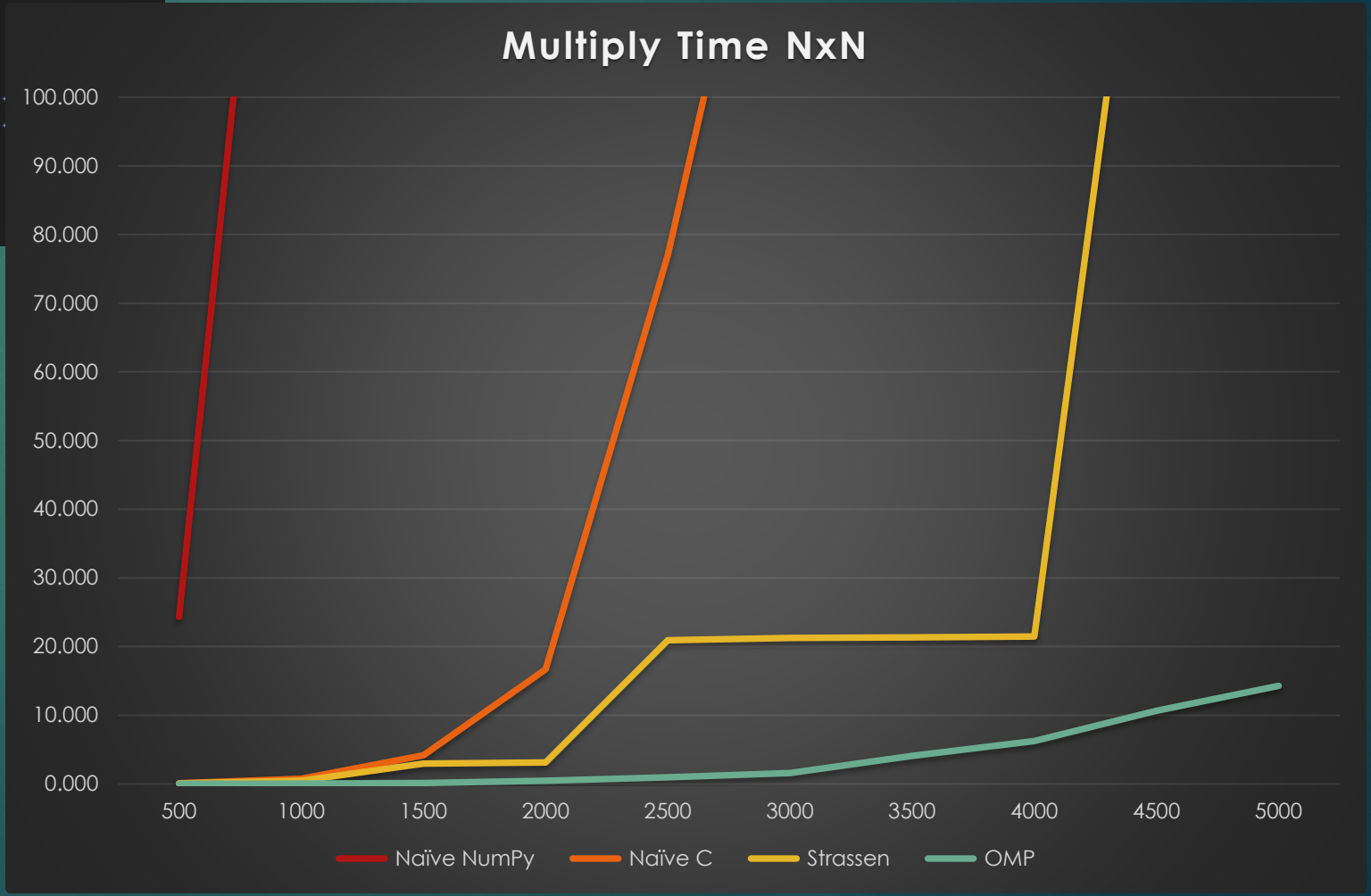
Multiply a Matrix - Strassen

```c
static void strassen(const double *A, const double *B, double *C, int m) {
    if (m <= THRESHOLD) {
        naive_mul(A, B, C, m);
        return;
    }

    int h = m / 2;
    size_t sz = (size_t)h * h;

    /*
     * Single allocation: 21 sub-matrices of size h×h.
     * Layout: A11 A12 A21 A22  B11 B12 B21 B22
     *         M1..M7  T1 T2  C11 C12 C21 C22
     */
    double *buf = malloc(21 * sz * sizeof(double));
    if (!buf) { fprintf(stderr, "OOM at m=%d\n", m); exit(1)

    double *A11=buf+ 0*sz, *A12=buf+ 1*sz, *A21=buf+ 2*sz, *
    double *B11=buf+ 4*sz, *B12=buf+ 5*sz, *B21=buf+ 6*sz, *
    double *M1 =buf+ 8*sz, *M2 =buf+ 9*sz, *M3 =buf+10*sz, *
    double *M5 =buf+12*sz, *M6 =buf+13*sz, *M7 =buf+14*sz;
    double *T1 =buf+15*sz, *T2 =buf+16*sz;
    double *C11=buf+17*sz, *C12=buf+18*sz, *C21=buf+19*sz, *

    extract(A, A11, m, 0, 0); extract(A, A12, m, 0, h);
    extract(A, A21, m, h, 0); extract(A, A22, m, h, h);
    extract(B, B11, m, 0, 0); extract(B, B12, m, 0, h);
    extract(B, B21, m, h, 0); extract(B, B22, m, h, h);

    /* M1 = (A11+A22)(B11+B22) */
    mat_add(A11, A22, T1, h); mat_add(B11, B22, T2, h);
    strassen(T1, T2, M1, h);
```



Multiply a Matrix - OMP

```
#pragma omp parallel for collapse(2) schedule(static)
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        double sum = 0.0;
        for (int k = 0; k < n; k++)
            sum += A[i * n + k] * BT[j * n + k
        C[i * n + j] = sum;
    }
```



Parallel Complexity

- Naïve Matrix multiplication is $O(N^3)$
  - N=2000 matrix => 8B ops
- Strassen Matrix multiplication is $O(N^{2.8074})$
  - N=2000 matrix => 2B ops
- Parallel Matrix multiplication *work* is $O(N^3)$
- Parallel Matrix multiplication *span* is $O(N)$*
  - N=2000 matrix => depth of ~2000

*simple case, assumptions vary

Parallel Complexity Basics

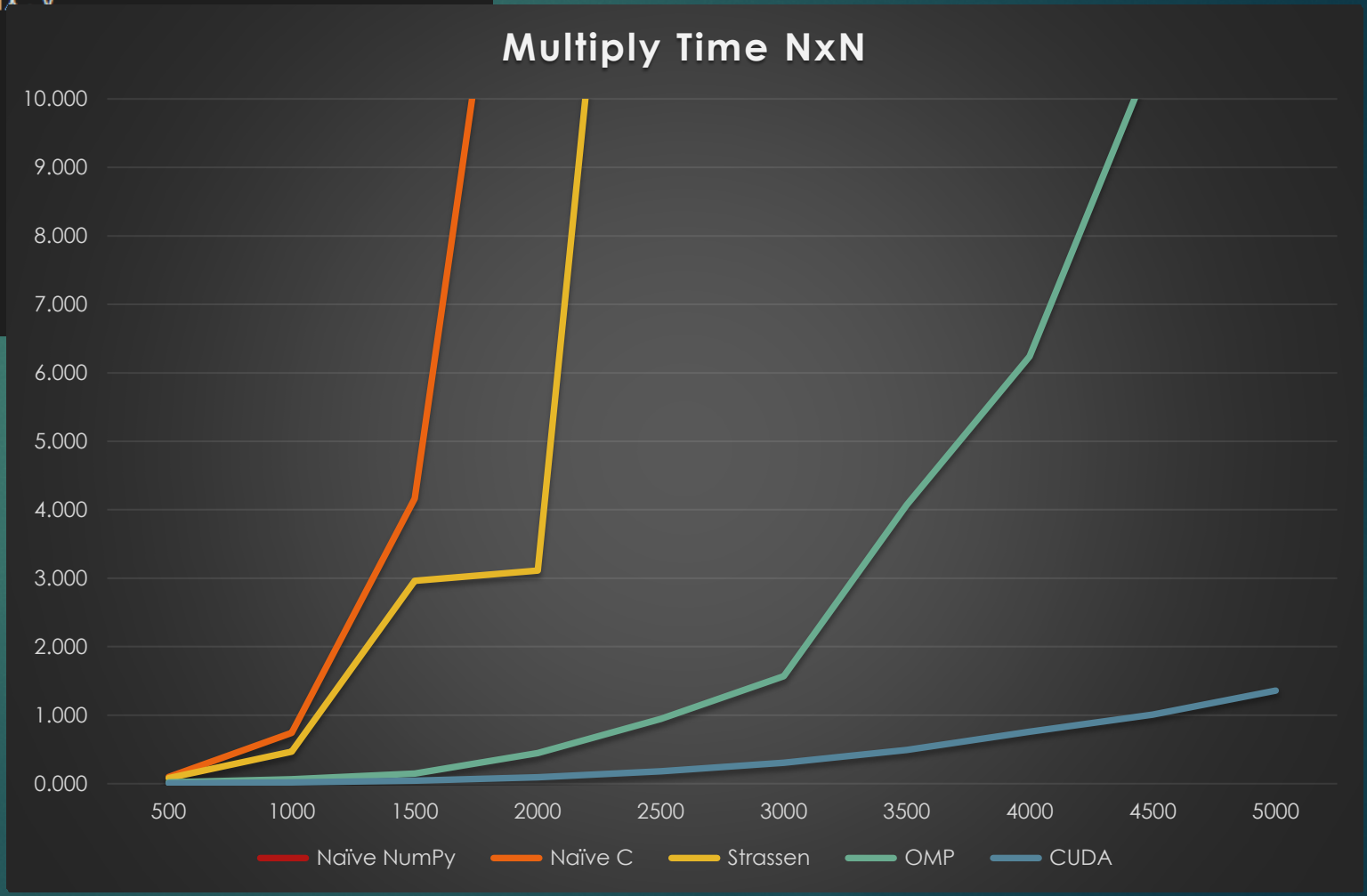
- T_1 is the time for the algorithm to execute on one processor
- T_P is the time on P processors
- T_∞ is for unlimited processors
  - the time required by the computation's critical path with unlimited parallelism
  - aka the depth of the computational DAG

- The Work Law: $T_P \geq T_1/P$ "No super-scaling"
- The Span Law: $T_P \geq T_\infty$ "No free lunch"

And many other results!

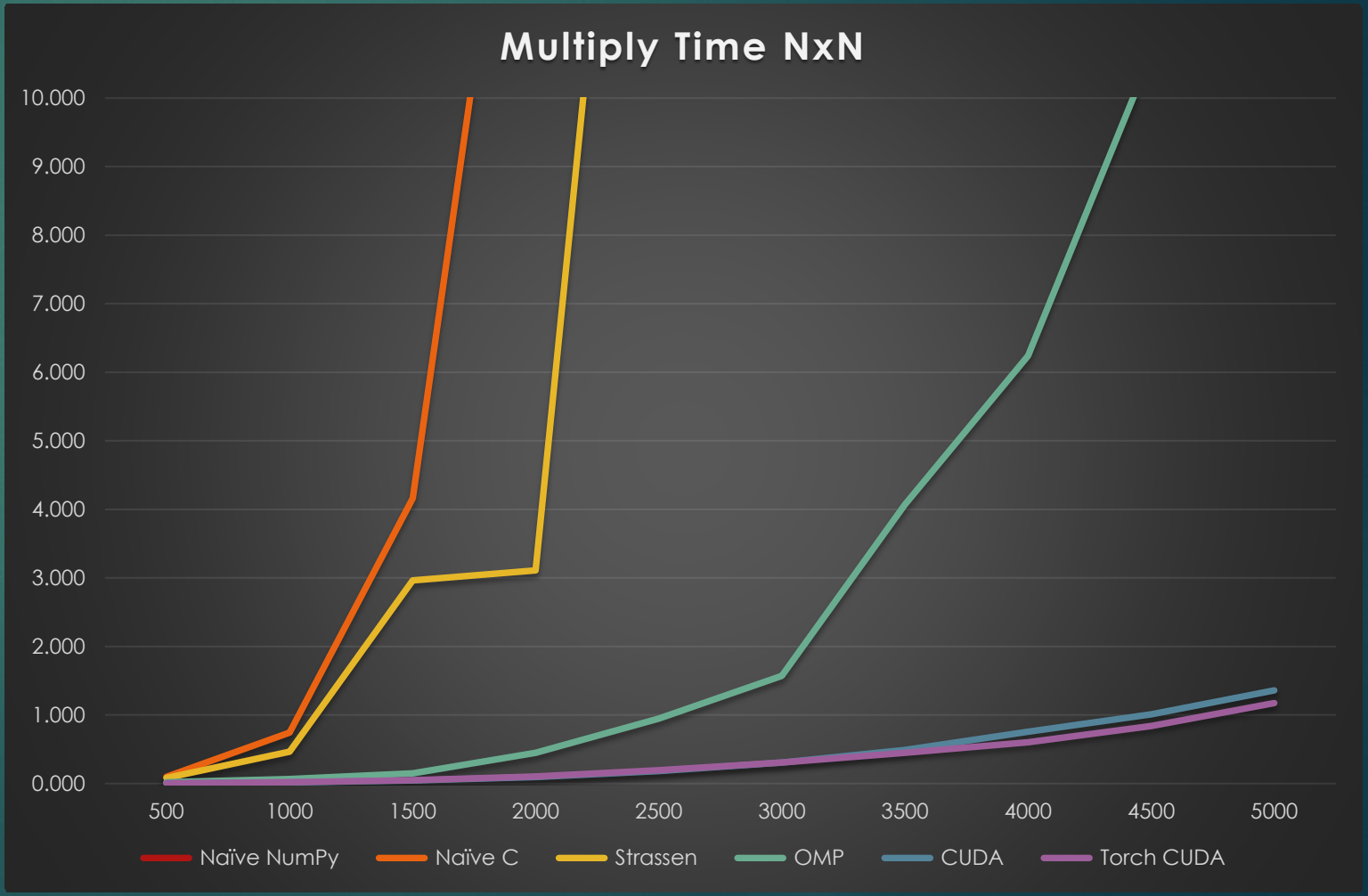
Multiply a Matrix - CUDA

```
__global__ static void multiply_kernel(const double *A, const double *B,
                                       double *C, int n) {
    int j = blockIdx.x * blockDim.x + threadIdx.x;
    int i = blockIdx.y * blockDim.y + threadIdx.y;

    if (i >= n || j >= n) return;

    double sum = 0.0;
    for (int k = 0; k < n; k++)
        sum += A[i * n + k] * B[k * n + j];
    C[i * n + j] = sum;
}
```



Multiply a Matrix - PyTorch

```
device = torch.device("cuda")

a = a_cpu.to(device)
b = b_cpu.to(device)
c = torch.matmul(a, b)
c_cpu = c.to("cpu")
torch.cuda.synchronize(device)
```



Multiply a Matrix – Real Big

- Actual supercomputers use distributed computation
- Hybrid clusters of CPU, GPU, and other accelerators
- Beyond work and span, need models to account for:

1. Power ⚡
2. Network bandwidth and latency 🖧

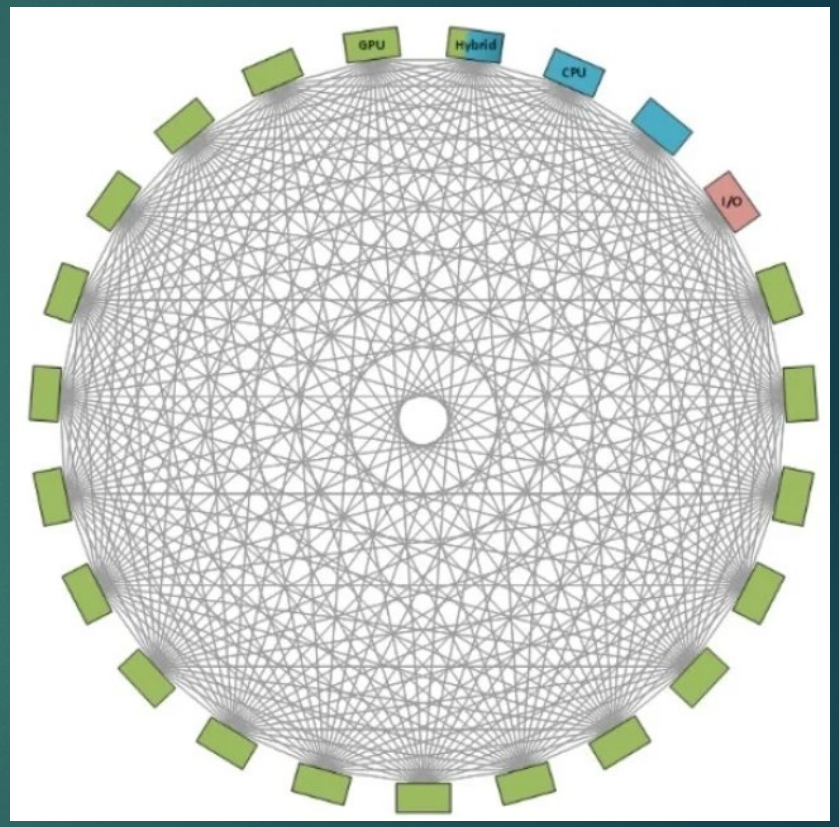
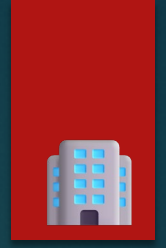


Image from: Turisini, M. & Cestari, Mirko & Amati, Giorgio. (2024). LEONARDO: A Pan-European Pre-Exascale Supercomputer for HPC and AI applications. Journal of large-scale research facilities JLSRF. 9. 10.17815/jlsrf-8-186.

Real World Challenge 2



- Fancy consultants know microservices are cool
- Energy Billing System
- To produce a single invoice required 1,750 network service requests
- Result: Process 4 bills per minute on $500K machine

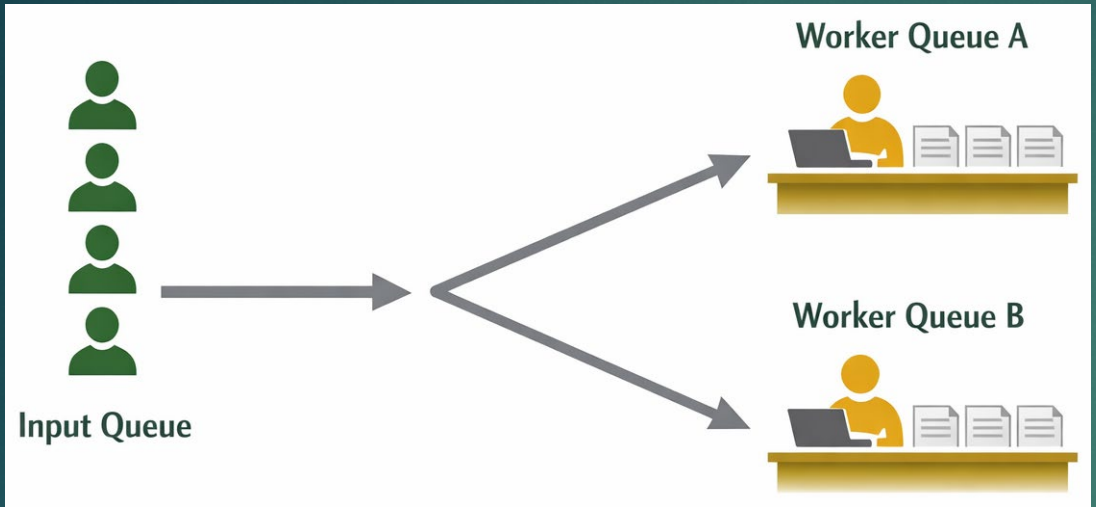
I/O Kills

Operation	Time in Microseconds ($1x10^{-6}$)
L1 cache	0.0005 µs
L2 cache	0.007
Main memory reference	0.1
Send 1K bytes on 1 Gpbs network	10.
Round trip within same datacenter	500.
Hard Disk Seek	10,000.
Send packet US->Europe->US	150,000.

Much of High Performance is about locality of data
To many software engineers don't understand this!

Adapted from: Latency Numbers Every Programmer Should Know
https://gist.github.com/jboner/2841832

Real World Challenge 3



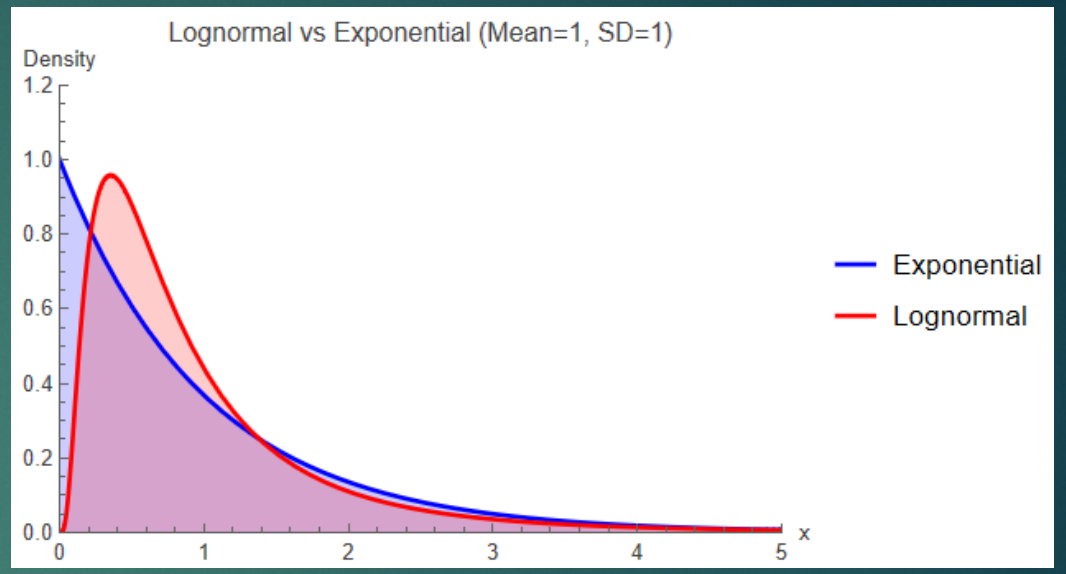
- Schedulers
- Web Traffic/Servers behind load balancers
- I/O Handlers
- Batches
- Call centers

Common assumptions:

- Uniform arrival rate
- Uniform handling rate
- As long as handling rate ≤ arrival rate, we're good

Queues

- Real-world arrivals often to follow an exponential distribution
  - Memoryless
- Real world service times often follow a lognormal distribution
  - Heavy-Tailed
- Result:




```
╔═ Queue Simulation Report
╠═ Configuration
  Servers (parallel queues) : 1
  Inter-arrival distribution: Uniform(90.0, 110.0)
  Service time distribution : Uniform(80.0, 100.0)
  Customers simulated       : 1000
  Simulation duration       : 100086.28
═ Wait Time  (time in queue before service)
  Mean   : 0.4757    Std Dev : 1.5469
  Median : 0.0000    Min     : 0.0000
  Max    : 11.7004    % waited: 13.1%
  95% CI (mean) : [0.3796, 0.5717]
```

```
╔═ Queue Simulation Report
╠═ Configuration
  Servers (parallel queues) : 1
  Inter-arrival distribution: Exponential(mean=100.0)
  Service time distribution : LogNormal(mean=90.0, std=2.0)
  Customers simulated       : 1000
  Simulation duration       : 102685.07
═ Wait Time  (time in queue before service)
  Mean   : 261.4886    Std Dev : 277.6886
  Median : 176.5037    Min     : 0.0000
  Max    : 1362.9354    % waited: 87.2%
  95% CI (mean) : [244.2480, 278.7291]
```

Same mean arrival
Same service times
Same server utilization
Same-ish duration

but mean wait times are 500x longer!

Queues

- Good news: reducing service time *or* adding servers has huge impact

```
╔═ Queue Simulation Report
╠═ Configuration
  Servers (parallel queues) : 1
  Inter-arrival distribution: Exponential(mean=100.0)
  Service time distribution : LogNormal(mean=45.0, std=2.0)
  Customers simulated       : 1000
  Simulation duration       : 97719.23
╠═ Wait Time  (time in queue before service)
  Mean   : 16.8975    Std Dev : 26.0586
  Median : 0.0000    Min     : 0.0000
  Max    : 156.0230    % waited: 44.5%
  95% CI (mean) : [15.2796, 18.5153]
```

```
╔═ Comparison: 1 vs 2 vs 4 Servers
  Metric                    1 server   2 servers   4 servers
  ──────────────────────────────────────────────────────────
  Avg wait time               313.73       10.61        0.89
  Avg service time             90.13       90.11       90.15
  Avg total time              403.86      100.72       91.03
  Max wait time              1237.72      211.12       73.43
```

Halve service time, average wait decreases by 96%

Double service unites, average wait decreases by 97%

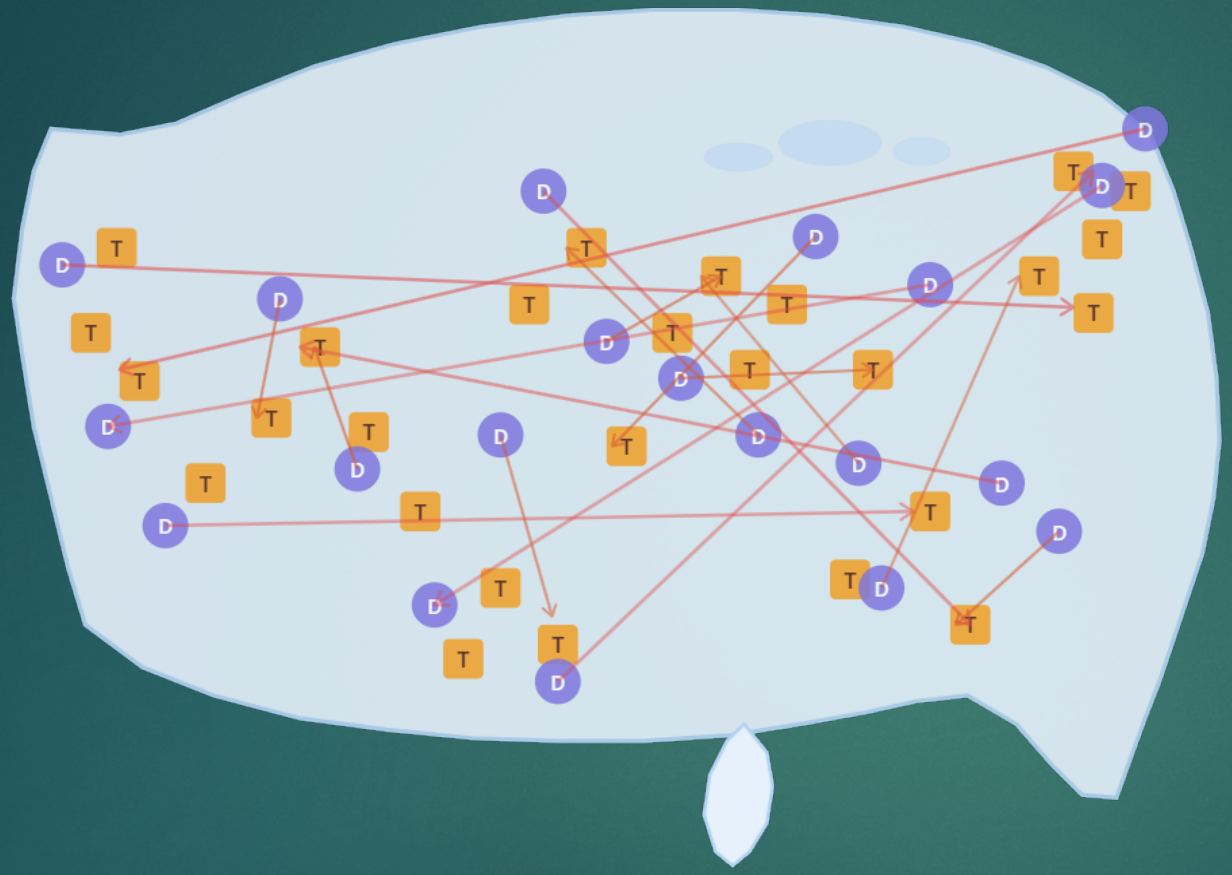
Bottom line: speed a queue, and you are a hero

Thousands of drivers. Thousands of trucks. Every assignment matters.

Real World Challenge 3

The scale problem

10,000 drivers × 10,000 trucks
= 100M possible pairings
Each with constraints:
HOS rules, CDL class, location



= Truck

= Driver

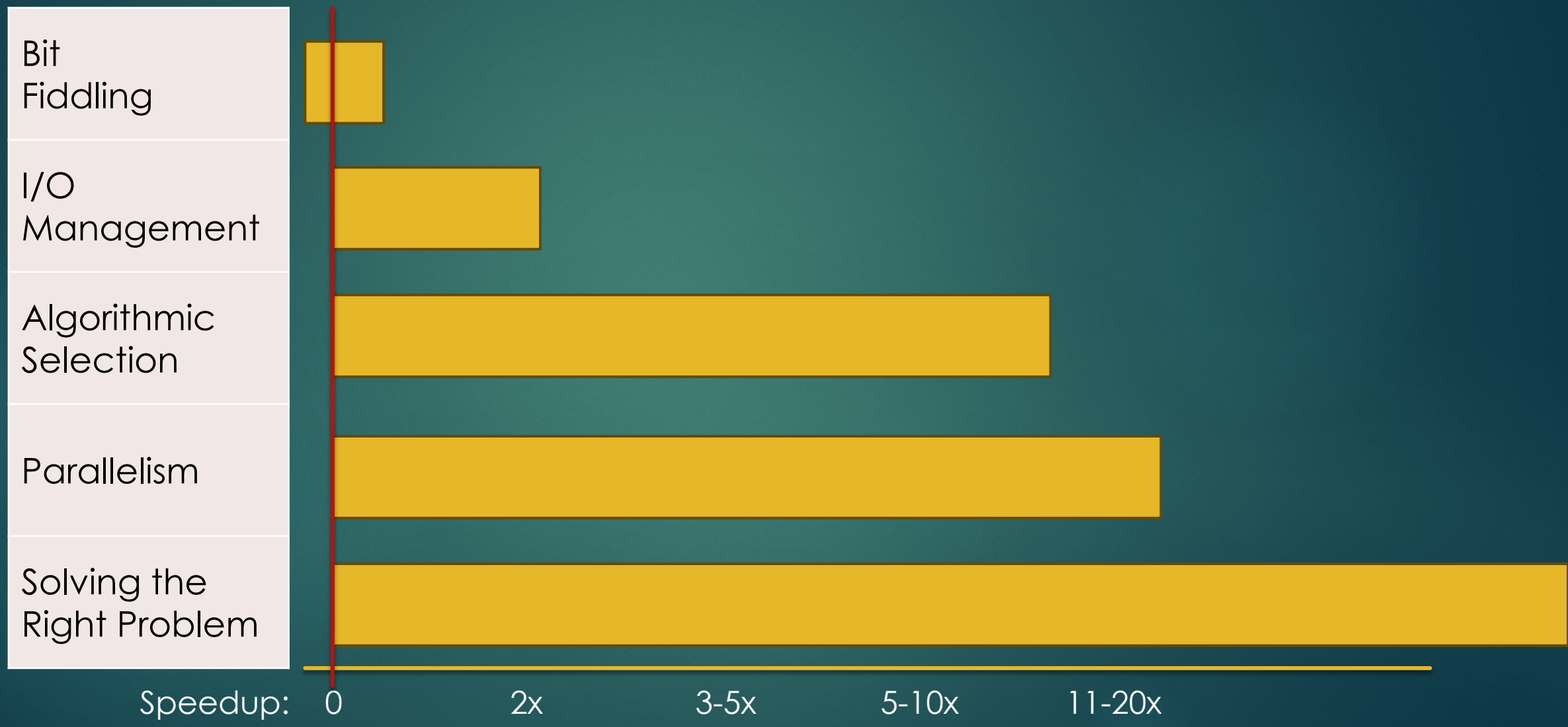
= Assignment

Crossing arrows = suboptimal assignments, wasted miles, delayed freight

When all else fails, compress the problem

- Heuristics for computationally intractable problems
- Shrink data sizes and types
  - The trend in AI LLMs
- Don’t solve for it at all

Where to Optimize?



Key Takeaways

✓ Work and Span, Work and Span

✓ Avoid bad I/O and Slow Queues

✓ Modern hardware is massively parallel — learn to exploit it

✓ Learn to love Linear Algebra

✓ Always keep learning

Seminar Code: https://github.com/hal-elrod/HPC-Talk
Learn HPC for free: https://sites.gatech.edu/omscsopencourseware/